

Section 1: Risk Identification from Vulnerability Analysis

The following critical risks were identified through the behavioral analysis of the `clientarm4n.elf` malware sample on an Ubuntu 20.04 system.

+1

Critical Risk 1: Unauthorized Data Exfiltration and C2 Communication

- **Explanation:** The analysis process (PID: 6231) repeatedly spawned shell commands (`curl` and `wget`) to communicate with external repositories, such as GitHub and Pastebin. This indicates an active attempt to establish Command & Control (C2) or exfiltrate system data.

+3

- **Treatment Recommendation: Mitigate.** The risk must be reduced by implementing strict outbound traffic controls.
- **Basic Mitigation Steps:**
 1. Implement **Egress Filtering** on the network firewall to block all unauthorized outbound connections to public code-sharing sites from production servers.
 2. Deploy an **Endpoint Detection and Response (EDR)** tool to automatically terminate processes that spawn shell commands for network fetching.

Critical Risk 2: Malicious Defense Evasion (Anti-Forensics)

- **Explanation:** The malware executed `rm -f` commands to delete temporary files in `/tmp/` immediately after execution. This technique is a confirmed **MITRE ATT&CK** technique (**T1070.004**) used to remove forensic evidence and hinder incident response.

+2

- **Treatment Recommendation: Mitigate.** System logging must be enhanced to ensure data is captured before deletion.
- **Basic Mitigation Steps:**
 1. Enable **Auditd** (Linux Auditing System) to log all file deletion events in the `/tmp/` directory to a centralized, write-only log server.
 2. Configure **Immutable Bit** attributes on critical system directories to prevent unauthorized deletion by non-root processes.

Section 2: Risk Monitoring Procedure

This procedure outlines how to track and monitor the risks identified above to ensure long-term system security.

- **Procedure Name:** Continuous Indicator & Behavior Monitoring (CIBM)

- **Monitoring Objective:** To identify recurrences of the `clientarm4n.elf` hash or its associated C2 behaviors across the network.
+1
 - **Tracking Steps:**
 1. **Weekly Log Review:** Security analysts will review firewall logs for any blocked connection attempts to the identified malicious URLs (e.g., the GitHub proxy list).
 2. **Automated Scanning:** Perform weekly automated file integrity scans across all Linux servers to check for the presence of the identified MD5 hash (`5ebfcae4fe2471fcc5695c2394773ff1`).
 3. **SIEM Dashboard:** Create a "Threat Behavior Dashboard" in a SIEM (like Wazuh or OpenCTI) that triggers a "High" alert if any process spawns a child process containing both `curl` and `/tmp/` paths.
 - **Justification:** This procedure is necessary because the malware demonstrated a high degree of automation. Monitoring for behavior (process spawning) rather than just static signatures (hashes) ensures protection against mutated versions of the threat.
-

Section 3: Documentation and Justification

All risk treatments were selected based on the **Joe Sandbox Classification** of "MALICIOUS" with a score indicating significant impact potential. The decision to prioritize **Evasion** and **C2** risks over other findings (like system discovery) is justified by the immediate threat these actions pose to data confidentiality and forensic integrity.